NHẬP MÔN TIN HỌC LẬP TRÌNH

CHƯƠNG 3-4 PYTHON

ThS. Nguyễn Văn Tiến + ThS. Huy Init

Nội dung

Phần 1: Giới thiệu về Python trong Google Colab

1. Thiết lập và sử dụng Google Colab
2. Chạy các tập lệnh Python đơn giản
3. Tìm hiểu về biến, vòng lặp và hàm (variables, loops, and functions)

Colab & Python

Python là một ngôn ngữ lập trình bậc cao, mã nguồn mở và đa năng với cú pháp đơn giản. Nó được dùng rộng rãi trong phát triển web, khoa học dữ liệu, trí tuệ nhân tạo và nhiều lĩnh vực khác.

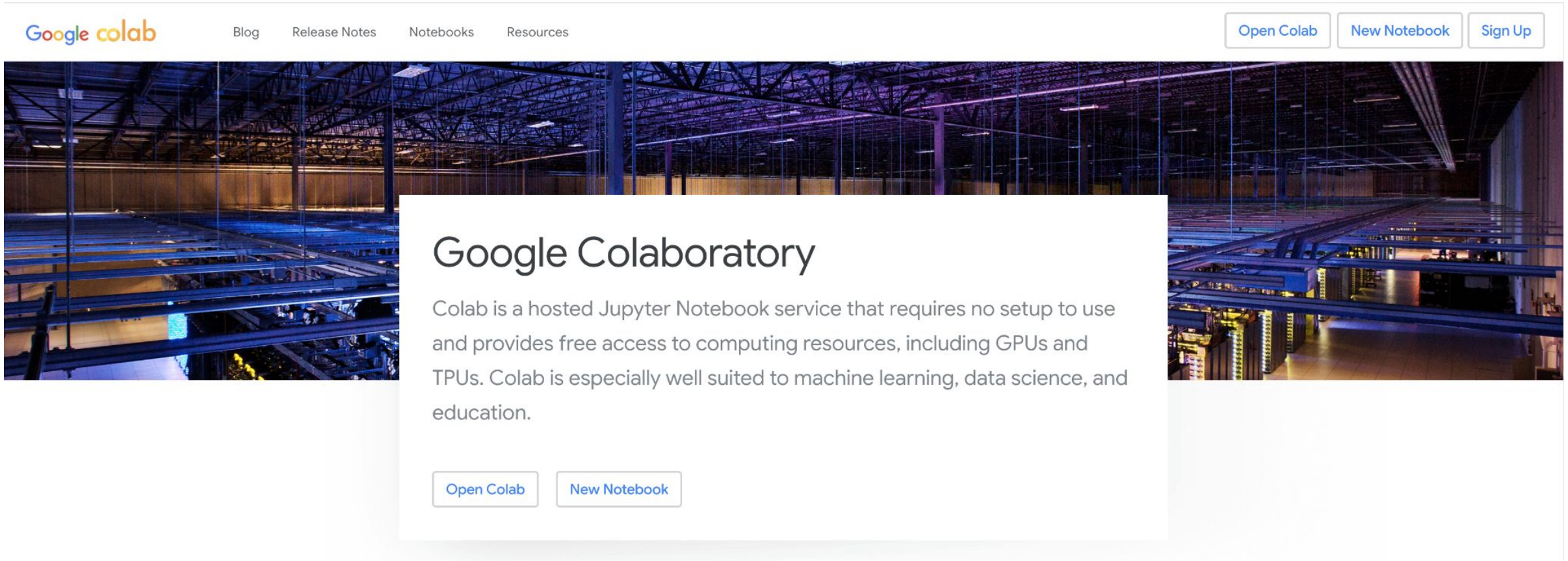
Để thực thi 1 chương trình được viết bằng Python, người dùng cần môi trường để viết và thực thi mã, có thể là trình biên dịch hoặc trình thông dịch Python cài đặt trên máy, hoặc sử dụng các nền tảng trực tuyến không cần cài đặt.

Colab & Python

Google Colab là một nền tảng trực tuyến miễn phí của Google cho phép người dùng viết và chạy mã Python ngay trên trình duyệt mà không cần cài đặt.

- Dễ sử dụng, không cần cài đặt.
- Hỗ trợ tài nguyên mạnh: Cung cấp miễn phí CPU, GPU và TPU.
- Chia sẻ và cộng tác thuận tiện.

Colab & Python



Cấu trúc cơ bản của một Python Script/Notebook

Google Colab Notebook gồm các phần chính giúp người dùng viết và thực thi mã Python hiệu quả trên trình duyệt, bao gồm các ô nhập mã gọi là Cell Code, các ô văn bản để giải thích hoặc ghi chú gọi là Cell Text, và nút Run để chạy từng ô mã lệnh hoặc toàn bộ notebook.

- Cell Code: Nơi người dùng nhập và chỉnh sửa mã Python để thực thi.
- Cell Text: Dùng để viết ghi chú, hướng dẫn hoặc mô tả bằng ngôn ngữ tự nhiên, thường theo định dạng markdown.
- Run: Nút hoặc lệnh để chạy mã trong Cell Code, cho phép kiểm tra kết quả ngay lập tức.

Cấu trúc cơ bản của một Python Script/Notebook



Chương trình máy tính và lệnh trong Python

Lệnh (Command)/Hướng dẫn (Instruction)

- Lệnh là thành phần cơ bản tạo nên chương trình, gồm từ khóa và tham số theo cú pháp cụ thể; tập hợp các lệnh mà CPU xử lý gọi là tập lệnh (instruction set).

Khối lệnh (Block of Code)

Các lệnh được nhóm thành khối (ví dụ hàm, phương thức). Thường được định nghĩa thành 1 đoạn có cùng cấp.

Chương trình máy tính và lệnh trong Python

Chương trình (Program) và Code

- Chương trình máy tính là tập hợp các hướng dẫn chi tiết, theo trình tự, để máy tính thực hiện một nhiệm vụ hoặc giải quyết vấn đề.
- Code là các dòng **lệnh** viết ra tạo thành chương trình, ban đầu dưới dạng mã nhị phân, ngày nay thường là các câu lệnh dễ đọc giống tiếng Anh.

Chạy Code/Chương trình (Running/Executing Code)

- Quá trình này là máy tính thực hiện từng hướng dẫn trong chương trình theo chu kỳ máy.

Biến (Variables) và Kiểu dữ liệu (Data Types)

Trong lập trình, Biến (Variables) và Kiểu dữ liệu (Data Types) là các thành phần cơ bản của mọi ngôn ngữ, bao gồm cả Python.

Biến

Biến là các vị trí bộ nhớ được đặt tên, dùng để lưu trữ dữ liệu tạm thời khi chương trình chạy và có thể thay đổi giá trị trong quá trình thực thi.

- Chức năng: Biến giống như "chiếc hộp" để lưu dữ liệu mà chương trình thao tác.
- Giá trị: Giá trị biến có thể thay đổi, khác với hằng số giữ nguyên giá trị cố định.
- Khai báo: Cần khai báo biến để hệ thống cấp phát bộ nhớ tương ứng cho biến trong chương trình.

Biến (Variables) và Kiểu dữ liệu (Data Types)

Kiểu dữ liệu cơ bản

Kiểu dữ liệu cơ bản mô tả loại dữ liệu được lưu trữ tại vị trí bộ nhớ giúp máy tính biết cách xử lý và phân bổ bộ nhớ phù hợp.

- Số nguyên (int).
- Số thực (float).
- Ký tự/Chuỗi (str).
- Logic/Boolean (bool).

Trong Python, biến không cần khai báo kiểu dữ liệu trước, giá trị gán sẽ quyết định kiểu, và có thể dùng hàm type() để kiểm tra kiểu dữ liệu của biến.

Biến (Variables) và Kiểu dữ liệu (Data Types)

Biến (Variables) và Kiểu dữ liệu (Data Types)

- Tên biến phải **bắt đầu** bằng **chữ cái (a-z, A-Z)** hoặc **dấu gạch dưới (_)** và **không** được bắt đầu **bằng số**.
- Tên biến **chỉ chứa** chữ cái, số và dấu gạch dưới, **không chứa** ký tự đặc biệt hoặc dấu cách.
- Tên biến **phân biệt** chữ hoa chữ thường (ví dụ: age, Age và AGE là ba biến khác nhau).
- Tên biến **không được trùng** với các **từ khóa** (keywords) của Python.
- Theo chuẩn PEP 8, nên dùng kiểu đặt tên **dạng chữ thường** và các từ cách nhau

Biến (Variables) và Kiểu dữ liệu (Data Types)

Ví dụ về tên biến đúng và sai trong Python:

Tên biến đúng: x, _age, total_sum, price123, name1

Tên biến sai:

- 2name (bắt đầu bằng số)
- total-sum (chứa dấu gạch ngang không hợp lệ)
- my var (chứa dấu cách)
- if (từ khóa của Python)
- @number (chứa ký tự đặc biệt không hợp lệ)

Lệnh nhập xuất dữ liệu

Câu lệnh nhập input()

- Chức năng: Thu thập dữ liệu người dùng nhập vào từ bàn phím.
- Cú pháp: bien = input("Câu thông báo cho người dùng").
- Lưu ý: Dữ liệu nhập vào luôn là kiểu chuỗi (string), ngay cả khi người dùng nhập số.

Lệnh nhập xuất dữ liệu

Câu lệnh xuất (print())

- Chức năng: Hiển thị thông tin (biến, chuỗi, giá trị) lên màn hình console.
- Cú pháp cơ bản: print(giá_trị_1, giá_trị_2, ...).
- Các tùy chỉnh:
  - Dấu phẩy: Ngăn cách các giá trị, hàm print() sẽ tự động thêm khoảng trắng giữa chúng.
  - Ký tự đặc biệt: Sử dụng \n để xuống dòng hoặc \t để tạo tab.
  - Định dạng f-string: Cách hiện đại và dễ đọc để chèn biến vào chuỗi.

Ví dụ:

```
ten = "John"
tuoi = 30
print(f"Tên tôi là {ten}, tôi {tuoi} tuổi.")
```

Lệnh nhập xuất dữ liệu

Đối số end: Thay đổi ký tự cuối cùng sau khi in. Mặc định là \n (xuống dòng).

- print("Đây là dòng 1", end=" ")
- print("Đây là dòng 2")

CÁC TOÁN TỬ CƠ BẢN

Các Toán tử (Operators) là các ký hiệu mã hóa (coding symbols) đại diện cho các hành động cơ bản của ngôn ngữ lập trình và là một trong những thành phần cốt lõi của tất cả các ngôn ngữ lập trình.

- Toán tử Số học (Arithmetic) dùng để thực hiện các phép tính toán học cơ bản như cộng, trừ, nhân và chia (ví dụ: +, -, *, /), đây là chức năng chính của Bộ phận Logic Số học (ALU) trong CPU.
- Toán tử Quan hệ (Relational) dùng để so sánh các giá trị, ví dụ: xác định một giá trị có lớn hơn, nhỏ hơn, hoặc bằng giá trị khác hay không.
- Toán tử Logic (Logical) (hoặc Boolean) dùng để kết hợp các điều kiện (như AND, OR, NOT) để tạo ra các quyết định hoặc truy vấn phức tạp.

Cấu trúc điều khiển

Cấu trúc điều khiển như:

- Trình tự thực thi
- Lựa chọn (if...else)
- Lặp lại (loop)

Cấu trúc điều khiển

Lệnh rẽ nhánh if/else là một cấu trúc điều khiển lựa chọn cho phép chương trình đưa ra quyết định (có/không), định hướng luồng thực thi để chọn một trong hai đường đi dựa trên việc liệu một điều kiện có đúng hay không.

Ví Dụ:

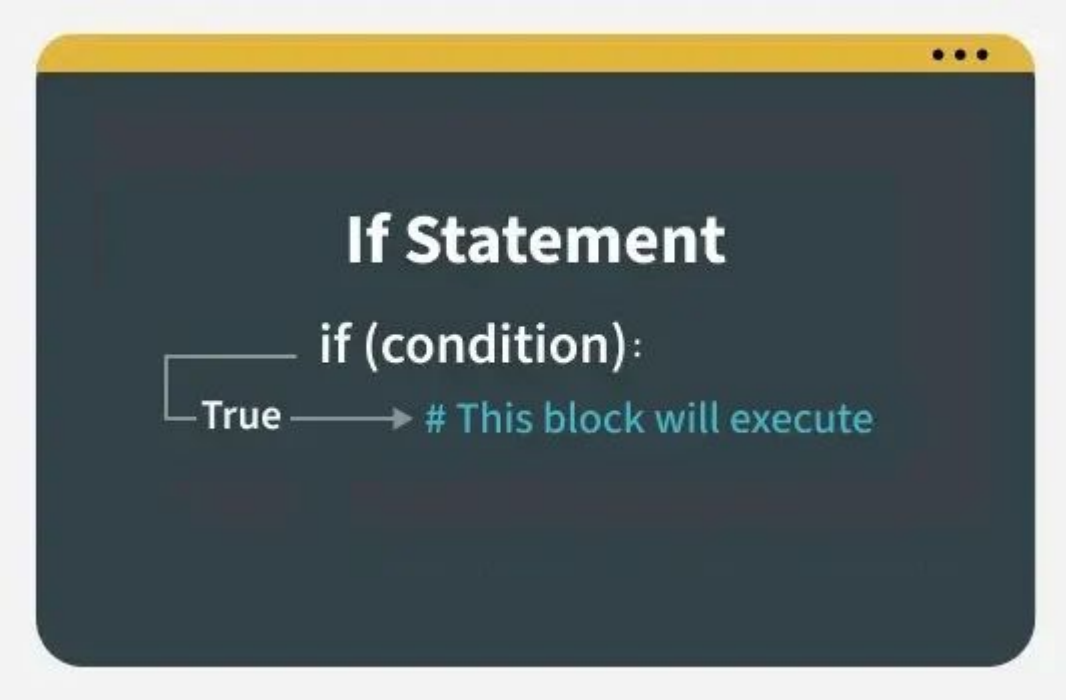
```
diem_so = 65
if diem_so >= 50:
    print ("Đậu")
else:
    print ("Rớt")
```

Cấu trúc điều khiển

Câu lệnh if

Cú pháp:

```
if điều_kiện:
    khối_lệnh_if
```



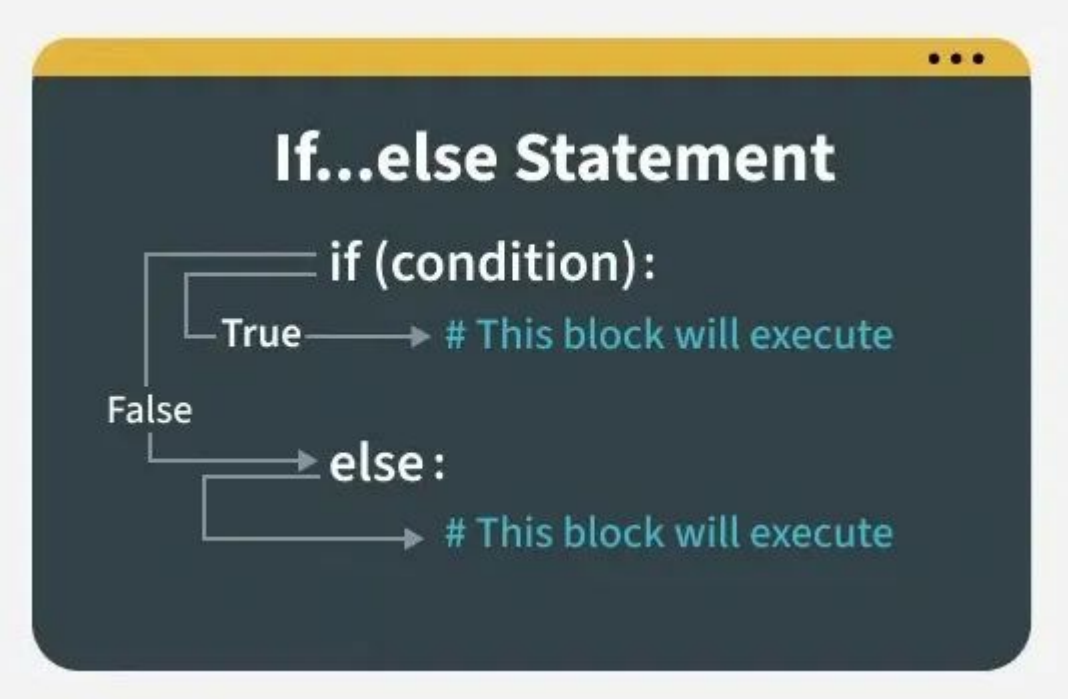
Ví dụ:

```
var = 10
if var > 5:
    print("Giá trị lớn hơn 5")
```

Cấu trúc điều khiển

Câu lệnh else
Cú pháp:

```
if điều_kiện:
    khối_lệnh_if
else:
    khối_lệnh_else
```



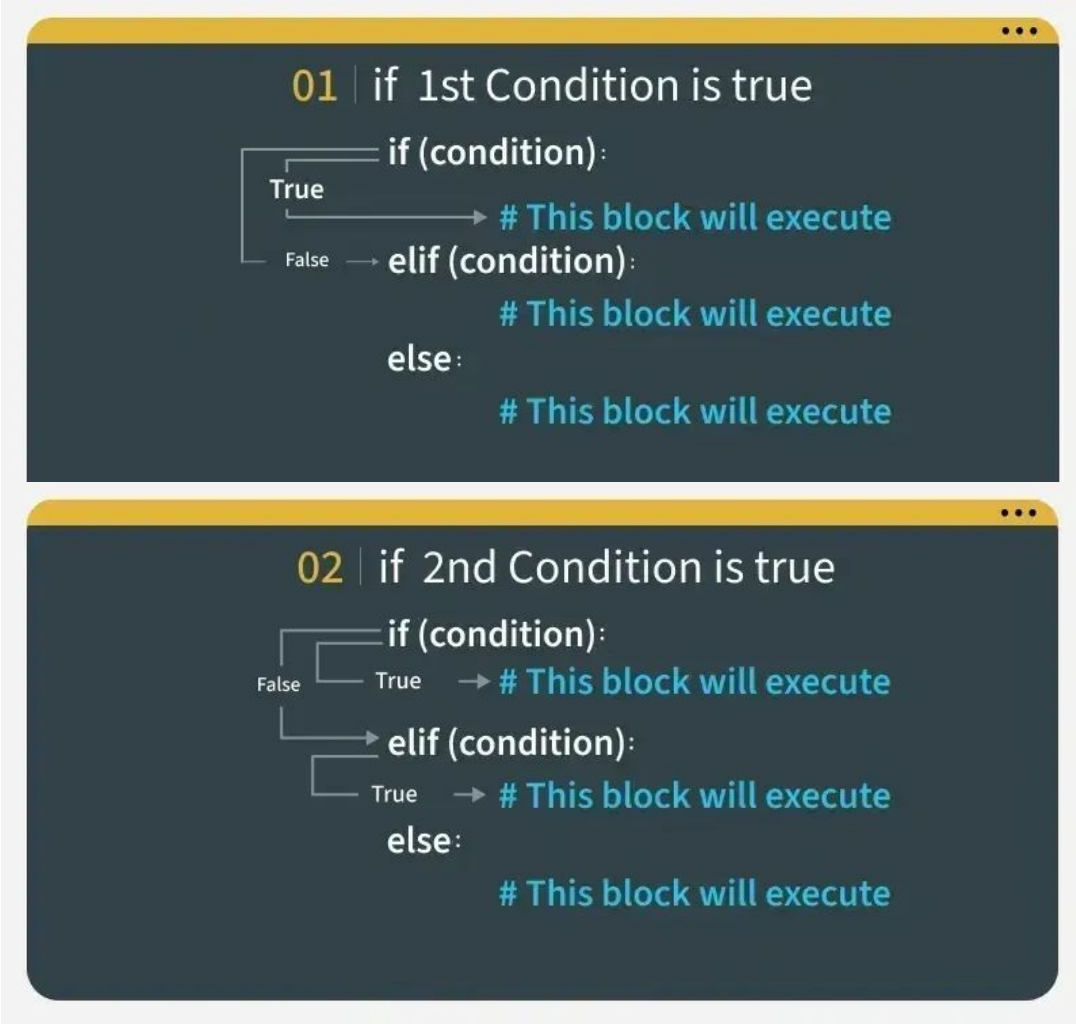
Ví dụ:

```
var = 10
if var > 5:
    print("Giá trị lớn hơn 5")
else:
    print("Giá trị nhỏ hơn hoặc bằng 5")
```

Cấu trúc điều khiển

Câu lệnh elif
Cú pháp:

```
if điều_kiện_1:
    khối_lệnh_1
elif điều_kiện_2:
    khối_lệnh_2
elif điều_kiện_3:
    khối_lệnh_3
...
else:
    khối_lệnh_else
```



Cấu trúc điều khiển

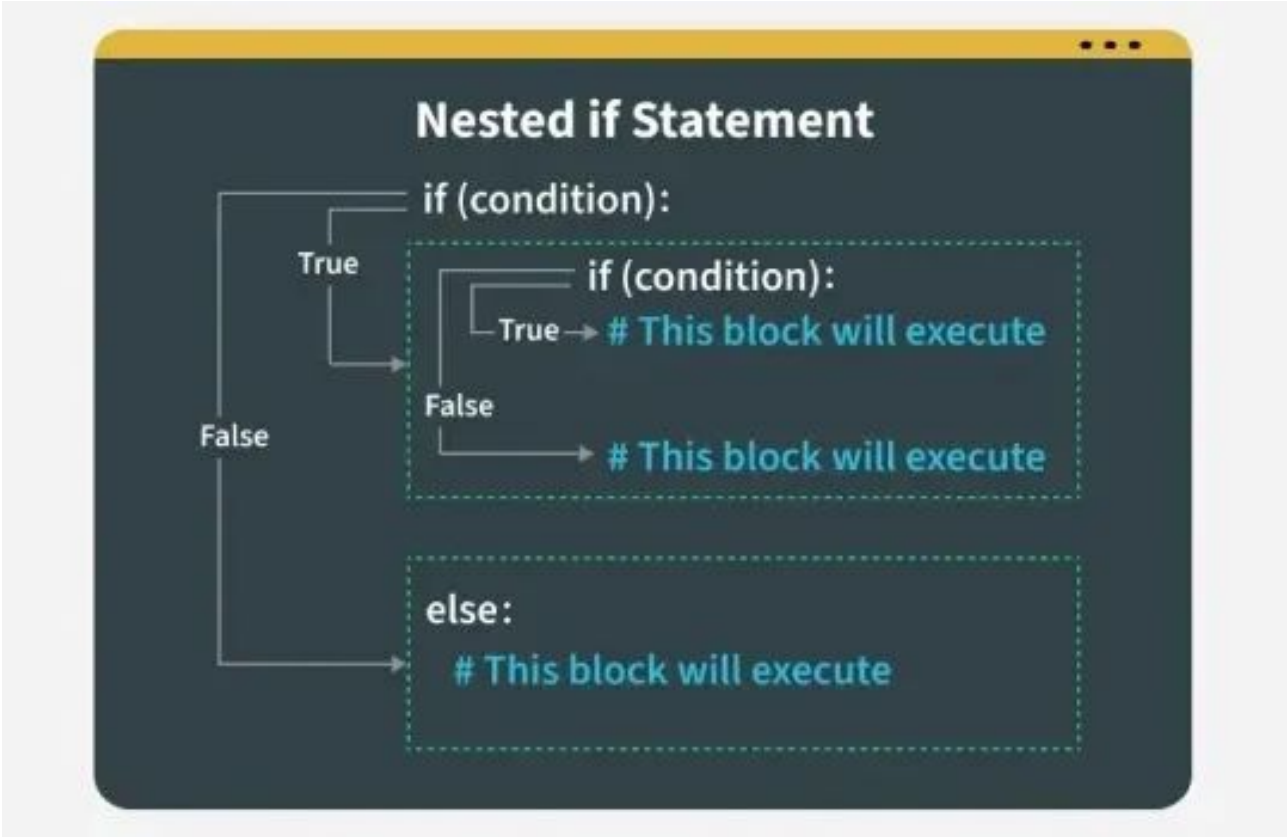
Ví dụ:

```
var = 100
if var == 200:
    print("1 - Nhận một giá trị true")
elif var == 150:
    print("2 - Nhận một giá trị true")
elif var == 100:
    print("3 - Nhận một giá trị true")
else:
    print("4 - Nhận một giá trị false")

print("Good bye!")
```

Cấu trúc điều khiển

Câu lệnh if lồng nhau



Cấu trúc điều khiển

Vòng lặp for và while là các cấu trúc điều khiển lặp lại, cho phép chương trình tự động lặp lại một chuỗi lệnh nhiều lần dựa trên một điều kiện kiểm tra.

Ví Dụ:

```
for i in range(3):
    print ("Xin chào thế giới lập trình.")

# Đầu ra (tương tự như ví dụ trong nguồn):
# Xin chào thế giới lập trình.
# Xin chào thế giới lập trình.
# Xin chào thế giới lập trình.
```

Cấu trúc điều khiển

Hàm range() trong Python được dùng để tạo ra một dãy số nguyên theo một phạm vi và bước nhảy xác định. Hàm này thường dùng trong các vòng lặp để lặp qua một dãy số liên tiếp.
Cú pháp cơ bản có hai dạng:

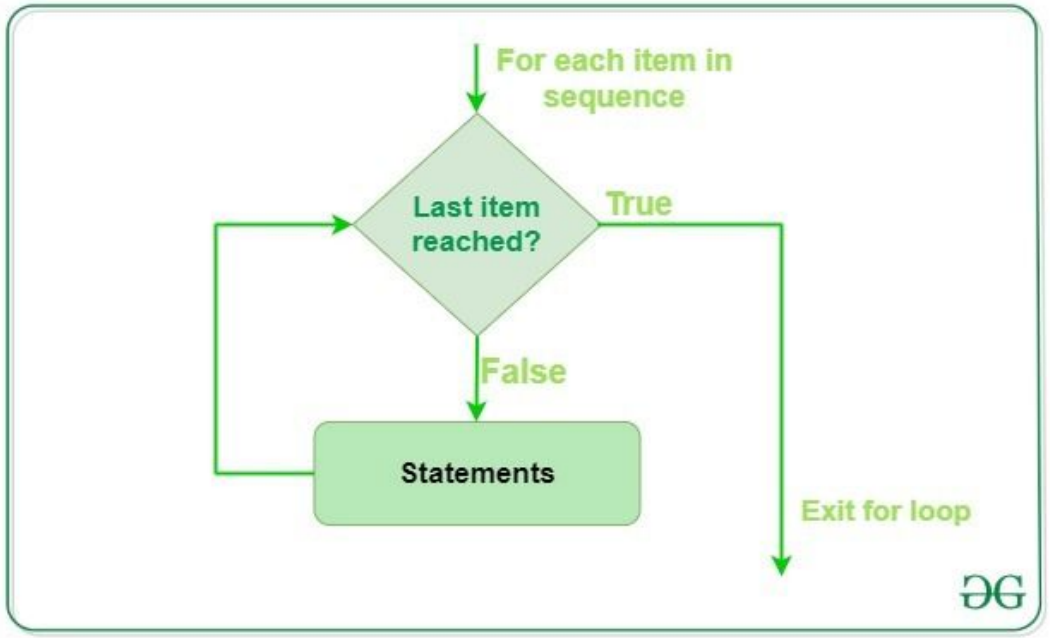
- range(stop) — tạo dãy số bắt đầu từ 0 đến stop-1
- range(start, stop[, step]) — tạo dãy số bắt đầu từ start đến stop-1 với bước nhảy step (mặc định là 1)

Tham số:

- start: điểm bắt đầu (mặc định 0)
- stop: điểm kết thúc (không bao gồm giá trị này)
- step: bước nhảy (có thể âm để đếm ngược)

Cấu trúc điều khiển

Vòng lặp for: Vòng lặp for được sử dụng để lặp qua một dãy như list (danh sách), tuple (bộ giá trị), string (chuỗi ký tự) hoặc range (phạm vi số). Nó cho phép thực thi một khối lệnh nhiều lần, mỗi lần tương ứng với một phần tử trong dãy.



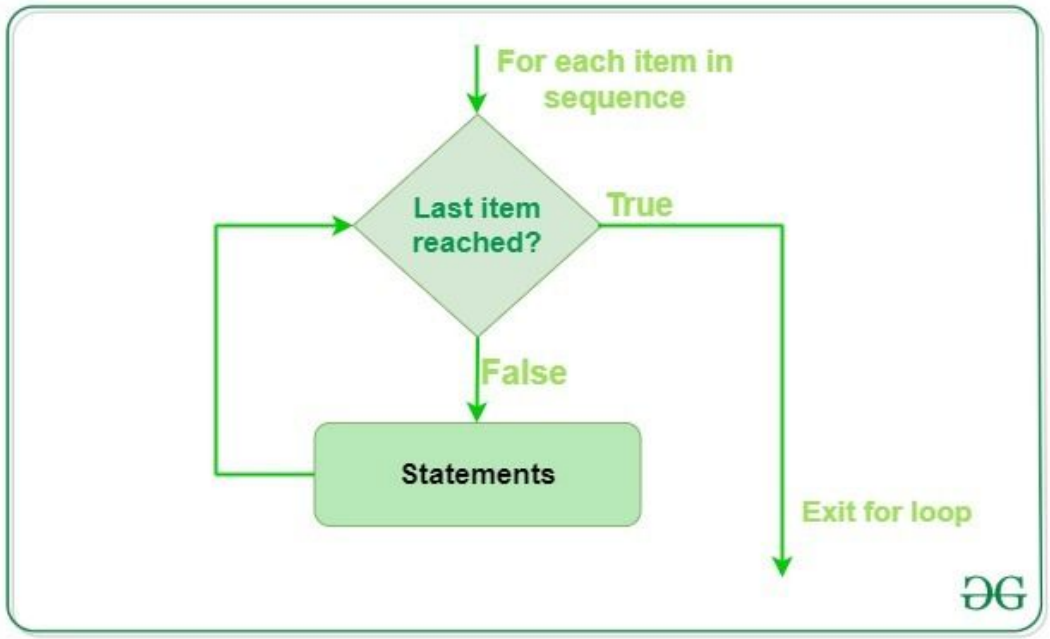
Cấu trúc điều khiển

Ví dụ:

```
n = 4
for i in range(0, n):
    print(i)
```

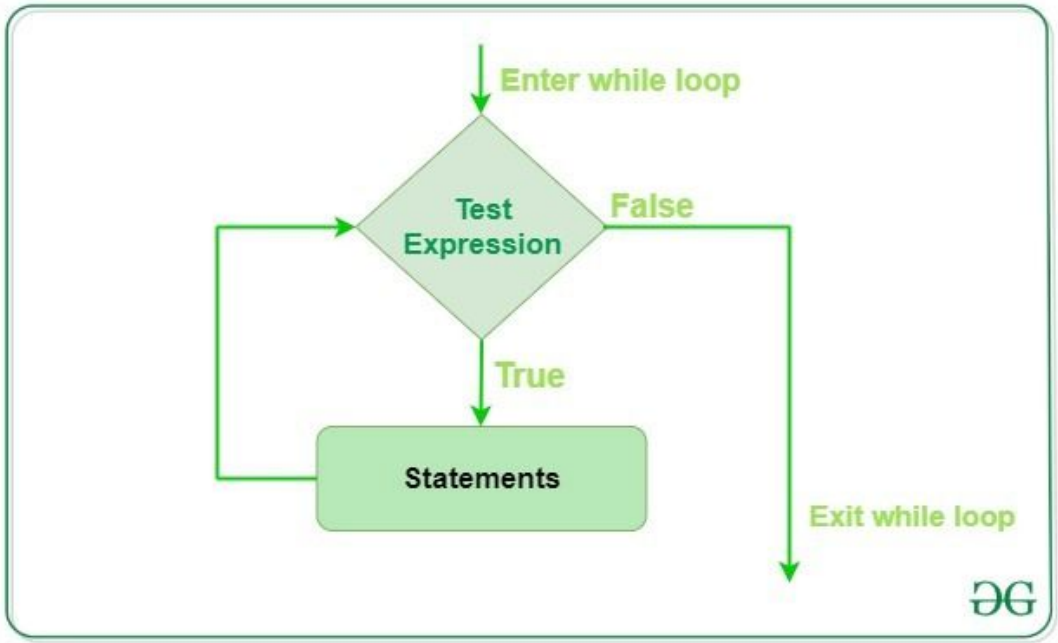
→ Output:

```
0
1
2
3
```



Cấu trúc điều khiển

Vòng lặp while: Vòng lặp while trong Python được sử dụng để thực thi một khối lệnh nhiều lần cho đến khi một điều kiện nhất định không còn đúng. Khi điều kiện trở thành sai, chương trình sẽ tiếp tục thực thi dòng lệnh ngay sau vòng lặp.



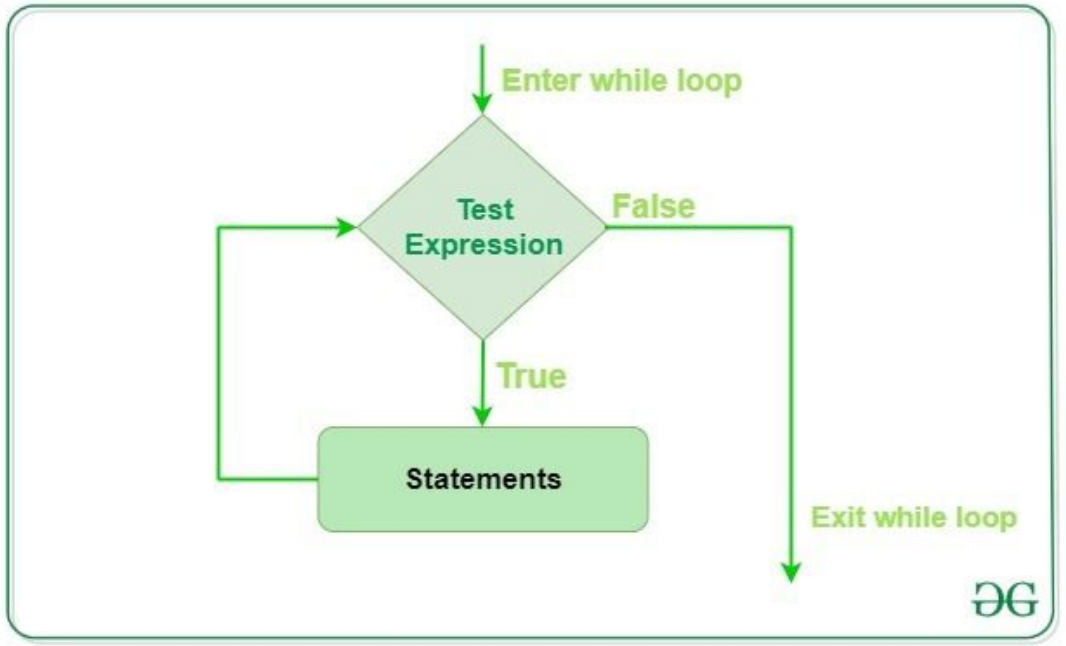
Cấu trúc điều khiển

Ví dụ:

```
cnt = 0
while (cnt < 3):
    cnt = cnt + 1
    print("Hello Geek")
```

→ tput:

```
Hello Geek
Hello Geek
Hello Geek
```



Cấu trúc điều khiển

Trong lập trình có thể thiết kế các vòng lặp lồng nhau. Ví dụ:

```
for i in range(1, 5):
    for j in range(i):
        print(i, end=' ')
    print()
```

Cấu trúc điều khiển

Câu lệnh điều khiển cấu trúc lặp: được sử dụng để thay đổi luồng thực thi thông thường của vòng lặp.

1. continue: Câu lệnh continue được sử dụng để bỏ qua lần lặp hiện tại của vòng lặp và chuyển sang lần lặp tiếp theo. Nó rất hữu ích khi ta muốn bỏ qua một số điều kiện nhất định mà không cần dừng toàn bộ vòng lặp.

2. break: được sử dụng để thoát khỏi vòng lặp sớm khi một điều kiện nhất định được thỏa mãn.

Cấu trúc điều khiển

Ví dụ:

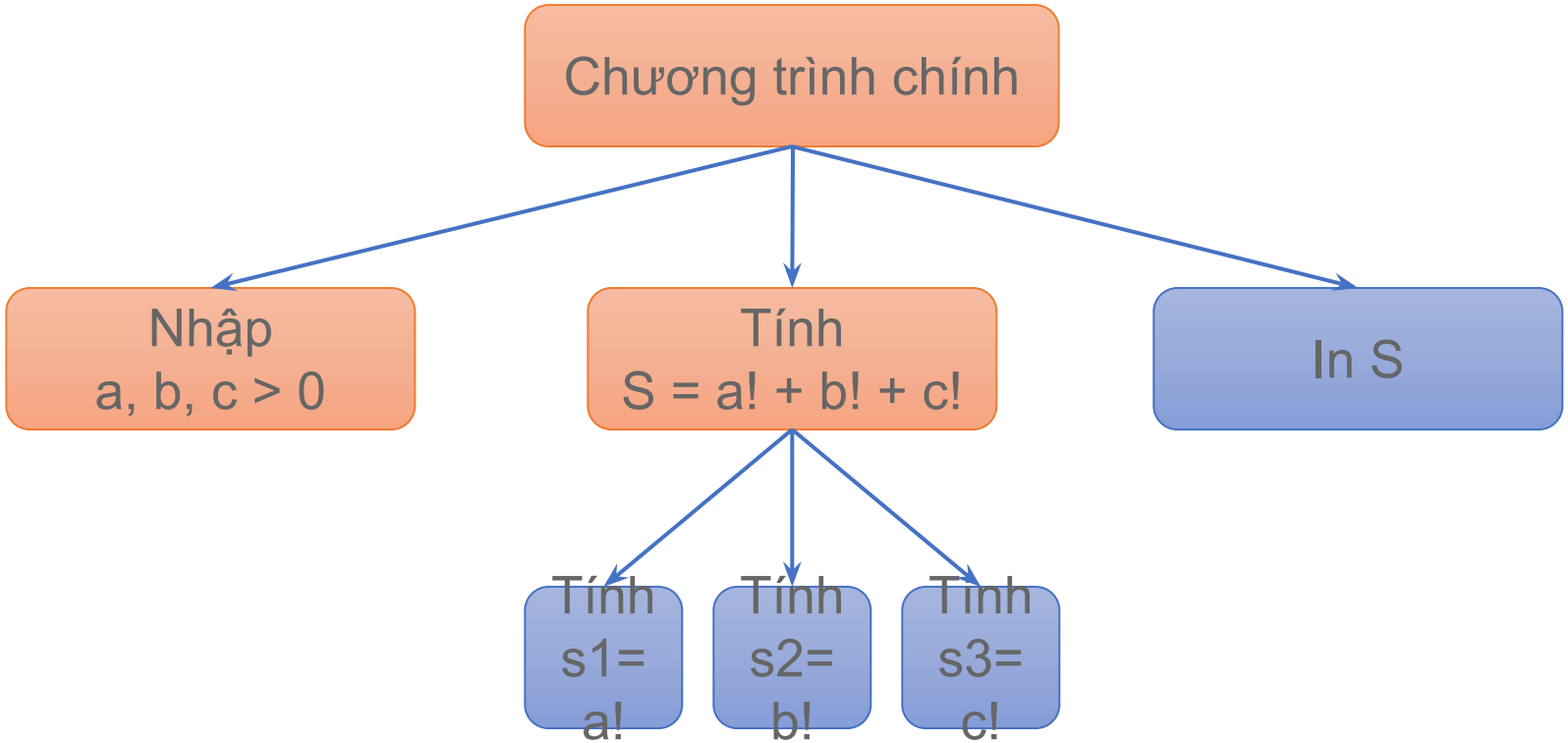
```
for i in range(10):
    if i == 5:
        break  # kết thúc vòng lặp ngay lập tức khi i bằng 5
    print(i)
print("Vòng lặp kết thúc do break.")

for i in range(10):
    if i == 5:
        continue  # bỏ qua phần còn lại của vòng lặp khi i bằng 5
    print(i)
print("Vòng lặp hoàn thành với continue.")
```

Đặt vấn đề - Hàm

Đặt vấn đề

Viết chương trình tính S = a! + b! + c! với a, b, c là các số nguyên dương nhập từ bàn phím.



Đặt vấn đề - Hàm

```
# Tính giai thừa a!
s1 = 1
for i in range(2, a + 1):
    s1 *= i
# Tính giai thừa b!
s2 = 1
for i in range(2, b + 1):
    s2 *= i
# Tính giai thừa c!
s3 = 1
for i in range(2, c + 1):
    s3 *= i
```

Đặt vấn đề - Hàm

Giải pháp => Viết 1 lần và sử dụng nhiều lần

Đoạn lệnh tính giai thừa tổng quát, n = a, b, c

```
s = 1
for i in range(2, n + 1):
    s *= i
```

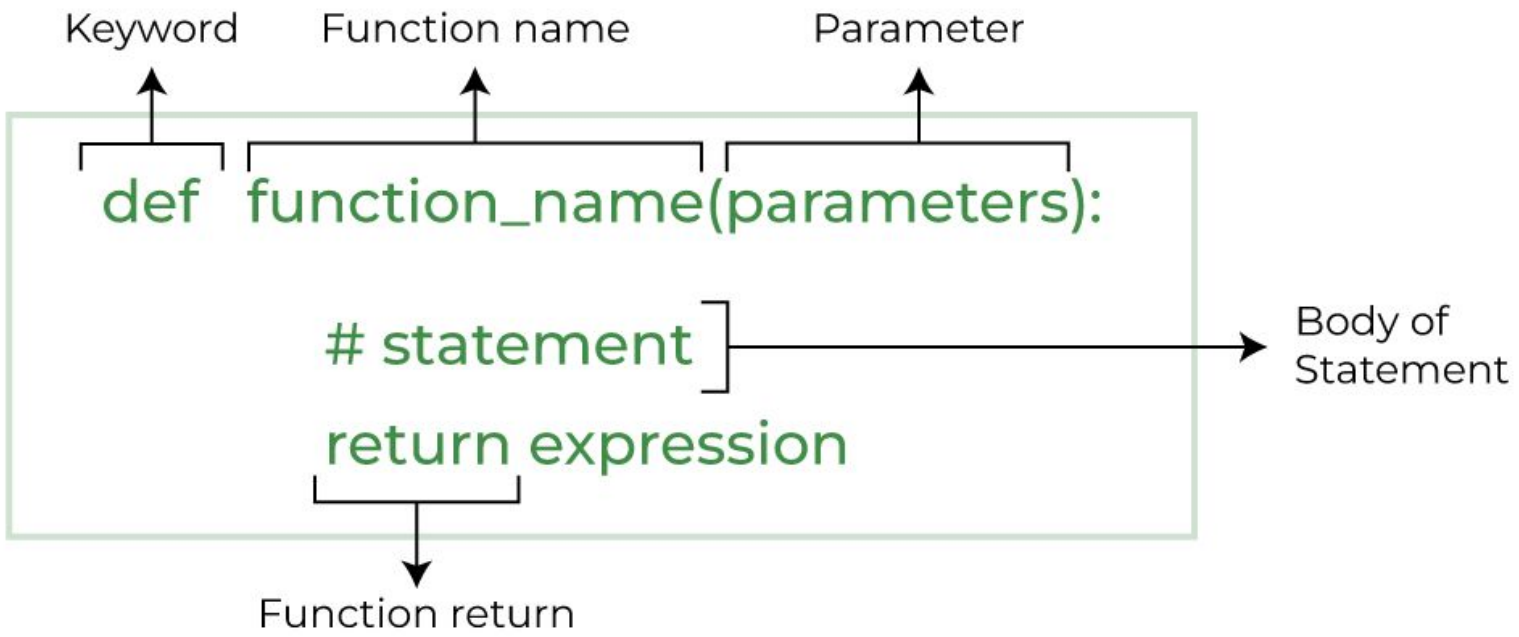
Cấu trúc điều khiển - Hàm

Hàm (Function) là các thành phần mã có thể tái sử dụng, giúp đóng gói một chuỗi lệnh không thuộc luồng thực thi chính, được định nghĩa bằng từ khóa def trong Python. Mục đích chính là để tái sử dụng logic code, giảm sự dư thừa, và làm cho chương trình trở nên dễ mở rộng, hiệu quả và dễ quản lý hơn.

Cấu trúc điều khiển - Hàm

Chúng ta có thể định nghĩa một hàm trong Python bằng cách sử dụng từ khóa def. Một hàm có thể nhận đầu vào dưới dạng các tham số.

Cú pháp để khai báo một hàm là:



Cấu trúc điều khiển - Hàm

Ví dụ: Định nghĩa một hàm bằng cách sử dụng def, hàm này sẽ in ra một thông điệp chào mừng khi được gọi.

```
def fun():
    print("Welcome to GFG")
```

Cấu trúc điều khiển - Hàm

Lời Gọi Hàm

Sau khi tạo một hàm trong Python, chúng ta có thể gọi nó bằng cách sử dụng tên của hàm, theo sau là dấu ngoặc chứa các tham số của hàm đó (nếu có).

Ví dụ:

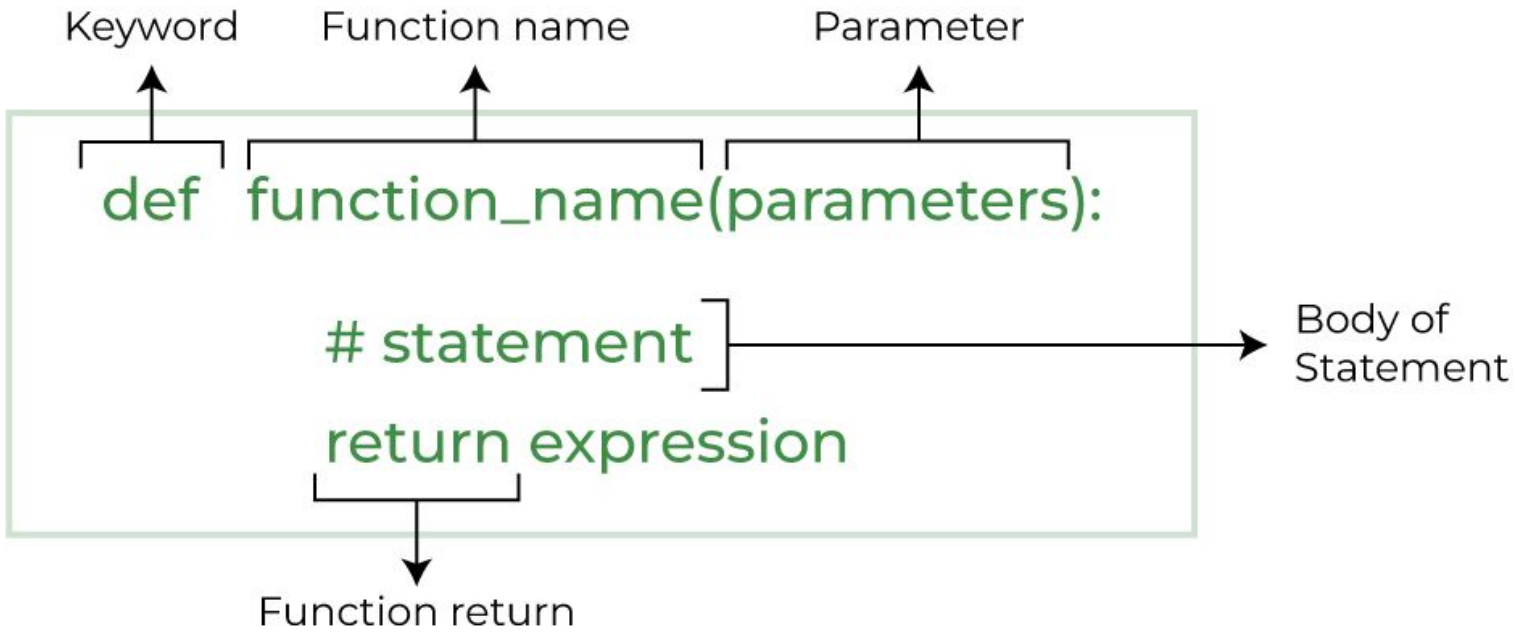
```
def fun():
    print("Welcome to GFG")

fun() # Driver code to call a function
```

Cấu trúc điều khiển - Hàm

Tham số Hàm

Tham số là các giá trị được truyền vào bên trong dấu ngoặc của hàm. Một hàm có thể có bất kỳ số lượng tham số nào, được phân tách bằng dấu phẩy.



Cấu trúc điều khiển - Hàm

Ví dụ: Hàm đơn giản trong Python để kiểm tra xem số được truyền vào hàm dưới dạng tham số là chẵn hay lẻ.

```
def evenOdd(x):
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"

print(evenOdd(16))
print(evenOdd(7))
```

Các Loại Tham Số Hàm

Các Loại Tham Số Hàm

1. Tham số Mặc định (Default Arguments)

Tham số mặc định là tham số sẽ nhận một giá trị mặc định nếu không có giá trị nào được cung cấp khi gọi hàm cho tham số đó.

Ví dụ:

```
def myFun(x, y=50):
    print("x: ", x)
    print("y: ", y)

myFun(10)
```

Các Loại Tham Số Hàm

Các Loại Tham Số Hàm
2. Tham số theo Tên (Keyword Arguments)
Với tham số theo tên, các giá trị được truyền bằng cách chỉ định rõ tên tham số, do đó thứ tự không quan trọng.

Ví dụ:

```
def student(fname, lname):
    print(fname, lname)

student(fname='Geeks', lname='Practice')
student(lname='Practice', fname='Geeks')
```

Các Loại Tham Số Hàm

3. Tham số theo Vị trí (Positional Arguments)
Với tham số theo vị trí, các giá trị được gán cho tham số dựa trên thứ tự xuất hiện trong lời gọi hàm.
Ví dụ:

```
def nameAge(name, age):
    print("Hi, I am", name)
    print("My age is ", age)

print("Case-1:")
nameAge("Suraj", 27)

print("\nCase-2:")
nameAge(27, "Suraj")
```

Câu lệnh return trong Hàm

Câu lệnh return kết thúc một hàm và gửi giá trị trở lại cho nơi gọi hàm. Nó có thể trả về bất kỳ kiểu dữ liệu nào, nhiều giá trị (được đóng gói trong tuple), hoặc None nếu không có giá trị nào được trả về.

Cú pháp: return [expression]

Ví dụ:

```
def square_value(num):
    """This function returns the square
    value of the entered number"""
    return num*num

print(square_value(2))
print(square_value(-4))
```

Câu lệnh return trong Hàm

Trong trường hợp hàm trả về nhiều giá trị

Cú pháp:

```
def ten_ham(tham_so1, tham_so2, ...):
    # các câu lệnh xử lý
    return gia_tri1, gia_tri2, gia_tri3, ...

# ---- Lời gọi hàm ----
bien1, bien2, bien3 = ten_ham(doi_so1, doi_so2, ...)
```

Câu lệnh return trong Hàm

Ví dụ:

```
def tinh_toan(a, b):
    tong = a + b
    hieu = a - b
    tich = a * b
    return tong, hieu, tich  # trả về 3 giá trị

t, h, ti = tinh_toan(8, 3)
print("Tổng:", t)
print("Hiệu:", h)
print("Tích:", ti)
```

Kiểu dữ liệu dạng tập hợp

Python cung cấp nhiều kiểu dữ liệu tập hợp để lưu trữ và quản lý dữ liệu hiệu quả.

- List (danh sách, có thứ tự, có thể thay đổi)
- Dictionary (Dict) (từ điển, lưu trữ cặp key-value)
- Set (tập hợp, các phần tử duy nhất, không có thứ tự)
- Tuple (bộ giá trị cố định, không thể thay đổi)

Kiểu dữ liệu dạng tập hợp - List

List là cấu trúc dữ liệu có thứ tự, cho phép lưu nhiều phần tử.

List là kiểu dữ liệu mutable: có thể thêm, sửa, xóa phần tử sau khi tạo.

Có thể chứa các kiểu dữ liệu khác nhau trong cùng một list.

Ví dụ: [1, 'a', 3.14, True]

Cách truy cập phần tử: dùng số thứ tự của nó, bắt đầu từ 0.

Cú pháp: <tên_list>[chỉ số].

Ví dụ: my_list = [10, 20, 30]

```
print(my_list[0])  # In ra 10
print(my_list[2])  # In ra 30
```

Kiểu dữ liệu dạng tập hợp - Dict

Dict lưu các cặp key-value, truy cập thông tin qua khóa (key).

Key phải là kiểu immutable và duy nhất trong dict.

Dict cũng là mutable, có thể thêm, sửa, xóa các cặp key-value.

Ví dụ: {'name': 'John', 'age': 25}

Truy cập giá trị bằng chìa khóa.

Cú pháp: <tên_dict>[tên khoá]

Ví dụ:
```
my_dict = {'name': 'Lan', 'age': 20}
print(my_dict['name'])  # In ra Lan
print(my_dict.get('age'))  # In ra 20
```

Kiểu dữ liệu dạng tập hợp - Set

Set là tập hợp các phần tử duy nhất, không theo thứ tự.
Set là mutable, cho phép thêm hoặc xóa phần tử.
Thường dùng để loại bỏ phần tử trùng hoặc kiểm tra nhanh sự tồn tại.
Ví dụ: {1, 2, 3}

Không thể truy cập phần tử bằng số thứ tự, nhưng có thể kiểm tra phần tử có trong set không.
Ví dụ: my_set = {1, 2, 3}

```
print(2 in my_set)  # In ra True
```

Kiểu dữ liệu dạng tập hợp - Tuple

Tuple giống list nhưng không thể thay đổi sau khi tạo.

Thứ tự phần tử được giữ nguyên.

Tuple được khai báo bằng dấu ngoặc (), phần tử cách nhau bởi dấu phẩy khác với list dùng [].

Ví dụ: (1, 2, 3) → khác với list dùng []

Cách truy cập phần tử cũng tương tự list.

Ví dụ: my_tuple = (5, 10, 15)

```
print(my_tuple[0])  # In ra 5
```

Mutable và Immutable

Phân biệt

- Mutable: có thể thay đổi sau khi tạo (List, Dict, Set).
- Immutable: không thể thay đổi (Tuple, String, Number).

→ Hiểu rõ sự khác biệt này giúp tránh lỗi khi truyền dữ liệu vào hàm con.

Mutable và Immutable với Hàm con

Trong Python, biến là các tham chiếu đến đối tượng. Khi truyền chúng vào hàm, hành vi sẽ phụ thuộc vào việc đối tượng có thể thay đổi (mutable, ví dụ: list, dict) hay không thể thay đổi (immutable, ví dụ: int, string, tuple). Nếu:

- Đối tượng Mutable: Thay đổi bên trong hàm sẽ ảnh hưởng đến đối tượng gốc.
- Đối tượng Immutable: Giá trị gốc không thay đổi.

→ Muốn thay đổi giá trị của tham số truyền vào thì gán giá trị của hàm trả về cho biến muốn thay đổi
Ví dụ: x = incre(x)

Câu lệnh return trong Hàm

```
def myFun(x):
    x[0] = 20

lst = [10, 11, 12, 13]
myFun(lst)
print(lst)   # list is modified

def myFun2(x):
    x = 20

a = 10
myFun2(a)
print(a)     # integer is not modified
```

Hàm đệ quy

Khái niệm

Một chương trình con có thể gọi một chương trình con khác.

Nếu gọi chính nó thì được gọi là sự đệ quy.

Số lần gọi này phải có giới hạn (điểm dừng)

Ví dụ

Tính S(n) = n! = 1*2*…*(n-1)*n

Ta thấy S(n) = S(n-1)*n

Vậy thay vì tính S(n) ta sẽ đi tính S(n-1)

Tương tự tính S(n-2), …, S(2), S(1), S(0) = 1

Hàm đệ quy

Ví dụ:

```
def factorial(n):
    if n == 0:
        return 1            # điều kiện dừng
    else:
        return n * factorial(n - 1)  # hàm tự gọi lại chính nó

print(factorial(4))
```

Hàm đệ quy

- Hàm đệ quy phải có 2 phần:

  + Phần dừng hay phải có trường hợp nguyên tố. Trong ví dụ ở trên thì trường hợp n=0 là trường hợp nguyên tố.

  + Phần đệ quy: là phần có gọi lại hàm đang được định nghĩa. Trong ví dụ trên thì phần đệ quy là n>0 thì n! = n * (n-1)!

- Sử dụng hàm đệ quy trong chương trình sẽ làm chương trình dễ đọc, dễ hiểu và vấn đề được nêu bật rõ ràng hơn. Tuy nhiên trong đa số trường hợp thì hàm đệ quy tốn bộ nhớ nhiều hơn và tốc độ thực hiện chương trình chậm hơn không đệ quy.
- Tùy từng bài có cụ thể mà người lập trình quyết định có nên dùng đệ quy hay không (có những trường hợp không dùng đệ quy thì không giải quyết được bài toán).

Kiểu dữ liệu String

Trong Python, string là một chuỗi các ký tự được đặt trong dấu ngoặc kép. Chuỗi có thể bao gồm chữ cái, chữ số, ký hiệu hoặc khoảng trắng. Vì Python không có kiểu dữ liệu ký tự riêng biệt nên ngay cả một ký tự đơn lẻ cũng được xem như một chuỗi có độ dài bằng một. String được sử dụng rộng rãi trong việc xử lý và thao tác văn bản.

Tạo một chuỗi (String)
Chuỗi có thể được tạo bằng dấu nháy đơn ('...') hoặc dấu nháy kép ("..."). Cả hai cách đều hoạt động giống nhau.

Ví dụ: a = “h" là 1 biến kiểu string
→ print (typeof(a))

Kiểu dữ liệu String

Chuỗi nhiều dòng (Multi-line Strings)

Sử dụng dấu nháy ba ('''...''') hoặc ("""...""") để tạo chuỗi trải dài trên nhiều dòng. Các ký tự xuống dòng sẽ được giữ nguyên.

Ví dụ: Định nghĩa và in ra chuỗi nhiều dòng bằng cả hai cách.

```
s = """I am Learning
Python String on GeeksforGeeks"""
print(s)

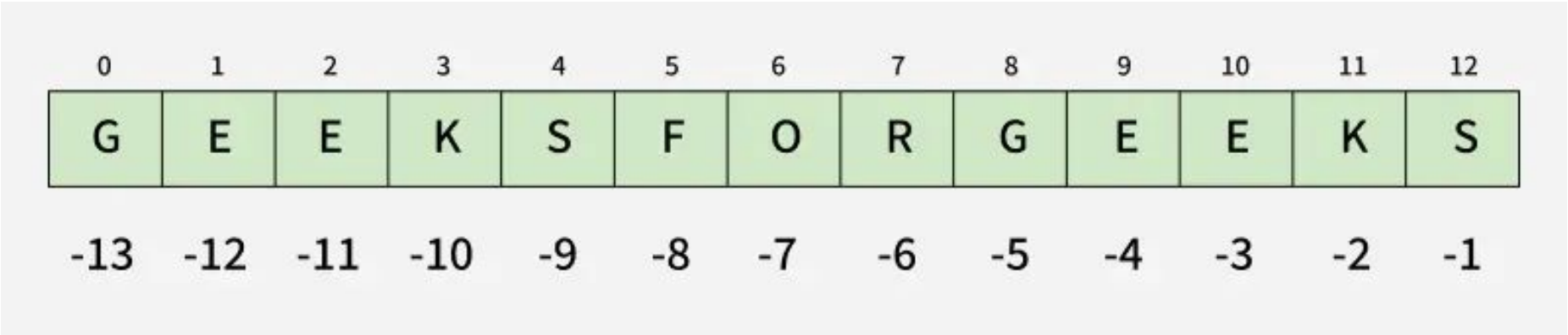
s = '''I'm a
Geek'''
print(s)
```

Truy xuất các phần tử

Chuỗi là một dãy có chỉ số (indexed sequence).

- Chỉ số dương bắt đầu từ 0 tính từ bên trái.
- Chỉ số âm bắt đầu từ -1 tính từ bên phải.

Điều này được minh họa trong hình dưới đây:



Truy xuất các phần tử

Để truy xuất các phần tử, ta sử dụng chỉ số theo cú pháp:

<tên biến> [chỉ số]

Ví dụ: Truy cập các ký tự cụ thể bằng chỉ số dương.

```
s = "GeeksforGeeks"
print(s[0])   # first character
print(s[4])   # 5th character
```

Đọc ký tự từ cuối chuỗi bằng chỉ số âm.

```
print(s[-10])   # ký tự thứ 3
print(s[-5])    # ký tự thứ 5 từ cuối
```

Cắt chuỗi

Cắt chuỗi (String Slicing)

Slicing là cách để trích xuất một phần của chuỗi bằng cách chỉ định chỉ số bắt đầu và chỉ số kết thúc.

Cú pháp: **string[start:end]**

Ví dụ:

```
s = "GeeksforGeeks"
print(s[1:4])    # characters from index 1 to 3
print(s[:3])     # from start to index 2
print(s[3:])     # from index 3 to end
print(s[::-1])   # reverse string
```

Duyệt chuỗi

Chuỗi là một kiểu dữ liệu có thể lặp (iterable), nghĩa là bạn có thể dùng vòng lặp để duyệt từng ký tự một.

Vòng lặp for lấy từng ký tự theo đúng thứ tự trong chuỗi, và mỗi lần lặp sẽ in ra ký tự kế tiếp.

Ví dụ:

```
s = "Python"
for char in s:
    print(char)
```

Tính bất biến của chuỗi

Chuỗi trong Python là immutable (bất biến), nghĩa là không thể thay đổi sau khi được tạo ra.
Nếu cần thao tác trên chuỗi, ta phải tạo một chuỗi mới bằng cách dùng nối chuỗi (concatenation), cắt chuỗi (slicing) hoặc định dạng chuỗi (formatting).
Trong ví dụ này, ta “thay đổi” ký tự đầu tiên bằng cách tạo một chuỗi mới dựa trên chuỗi gốc:

```
s = "geeksforGeeks"
s[0] = “G" → không hoạt động
s = "G" + s[1:]   # tạo chuỗi mới
print(s)
```

Các hàm làm việc với string thông dụng

1. len()

Hàm len() trả về tổng số ký tự trong chuỗi (bao gồm cả khoảng trắng và dấu câu).

Ví dụ:

```
s = "GeeksforGeeks"
print(len(s))
```

Các hàm làm việc với string thông dụng

2. upper() và lower()

- upper() chuyển toàn bộ ký tự thành chữ hoa.
- lower() chuyển toàn bộ ký tự thành chữ thường.

Ví dụ:

```
s = "Hello World"
print(s.upper())
print(s.lower())
```

Các hàm làm việc với string thông dụng

3. strip() và replace()

- strip() loại bỏ khoảng trắng ở đầu và cuối chuỗi.
- replace() thay thế tất cả các chuỗi con bằng chuỗi mới.

Ví dụ:

```
s = "   Gfg   "
print(s.strip())

s = "Python is fun"
print(s.replace("fun", "awesome"))
```

Các hàm làm việc với string thông dụng

Nối chuỗi và lặp chuỗi

Ta có thể nối chuỗi bằng toán tử + và lặp chuỗi bằng toán tử *.

1. Nối chuỗi bằng +

Ví dụ: Nối hai từ với khoảng trắng.

```
s1 = "Hello"
s2 = "World"
print(s1 + " " + s2)
```

Các hàm làm việc với string thông dụng

Nối chuỗi và lặp chuỗi

Ta có thể nối chuỗi bằng toán tử + và lặp chuỗi bằng toán tử *.

2. Lặp chuỗi bằng *

Ví dụ: Lặp lời chào 3 lần.

```
s = "Hello "
print(s * 3)
```

Định dạng chuỗi

Định dạng chuỗi (Formatting Strings)

Python có nhiều cách để đưa biến vào trong chuỗi.

1. Sử dụng f-string

Ví dụ:

```
name = "Alice"
age = 22
print(f"Name: {name}, Age: {age}")
```

Định dạng chuỗi

Định dạng chuỗi (Formatting Strings)

Python có nhiều cách để đưa biến vào trong chuỗi.

2. Sử dụng format()

Ví dụ:

```
s = "My name is {} and I am {} years old.".format("Alice", 22)
print(s)
```

Kiểm tra thành viên trong chuỗi

Từ khóa **in** kiểm tra xem một chuỗi con có tồn tại trong chuỗi hay không.

Ví dụ:

```
s = "GeeksforGeeks"
print("Geeks" in s)
print("GfG" in s)
```

So sánh chuỗi trong Python

Python hỗ trợ nhiều toán tử để so sánh chuỗi : ==, !=, <, <=, >, >=

Các toán tử này cho phép kiểm tra bằng nhau và so sánh thứ tự từ điển.

Ví dụ:

```
s1 = "apple"
s2 = "banana"
# "apple" không bằng "banana"; "apple" khác "banana";  "apple"
# đứng trước "banana" theo thứ tự từ điển
print(s1 == s2)
print(s1 != s2)
print(s1 < s2)
```

So sánh chuỗi

So sánh không phân biệt chữ hoa chữ thường

Ví dụ:

```
s1 = "Apple"
s2 = "apple"

# So sánh không phân biệt hoa/thường
print(s1.lower() == s2.lower())
```

So sánh tiền tố hậu tố

Sử dụng startswith() và endswith()

```
s = "hello world"
# Kiểm tra chuỗi bắt đầu bằng "hello"
print(s.startswith("hello"))

# Kiểm tra chuỗi kết thúc bằng "world"
print(s.endswith("world"))
```

Một số thao tác khác

1. Truyển đổi từ string -> int:

```
s = "42"
num = int(s)
print(num)
```

2. Truyển đổi từ int -> string

```
n = 42
s = str(n)
print(s)
```

Một số thao tác khác

3. Kiểm tra giá trị

Phương thức str.isdigit() kiểm tra xem một chuỗi có chỉ chứa các chữ số hay không trước khi chuyển đổi sang số. Phương pháp này đảm bảo rằng dữ liệu nhập vào chỉ bao gồm các chữ số, tránh lỗi khi ép kiểu.

Ví dụ:

```
s = "12345"
if s.isdigit():
    num = int(s)
    print(num)
else:
    print("The string is not numeric.")
```

Phần 2: Lập trình với Hỗ trợ AI

1. Sử dụng ChatGPT và Copilot để tạo mã
2. Sửa đổi và gỡ lỗi mã với sự hỗ trợ của AI
3. Viết các chương trình nhỏ để tính toán kinh tế

Sử dụng ChatGPT và Copilot để tạo mã

Để sử dụng AI sinh mã code hiệu quả cần tập trung vào việc tạo ra các lệnh đầu vào (prompt) rõ ràng, cụ thể, có định hướng và cấu trúc phù hợp nhằm giúp AI hiểu đúng yêu cầu.

- Rõ ràng và cụ thể: Mô tả nhiệm vụ rõ ràng, tránh mơ hồ để AI không bị hiểu sai hoặc trả lời lan man.
- Cung cấp ngữ cảnh đầy đủ: Đưa đủ thông tin và bối cảnh để AI có cơ sở xử lý và tạo mã chính xác hơn.
- Chỉ định định dạng và ví dụ: Xác định kiểu đầu ra mong muốn (code, bảng, list) và thêm ví dụ minh họa để tăng độ tin cậy kết quả.

Sử dụng ChatGPT và Copilot để tạo mã

Bài toán tính chiết khấu có thể phát biểu như sau:

Cho biết giá gốc của sản phẩm và tỷ lệ phần trăm chiết khấu, hãy tính toán giá cuối cùng sau khi áp dụng chiết khấu đó.

→ Prompt tham khảo: "Viết đoạn mã Python nhận giá gốc và phần trăm chiết khấu làm đầu vào, tính và in giá cuối cùng sau khi áp dụng chiết khấu."

Sử dụng ChatGPT và Copilot để chỉnh sửa mã

Điều chỉnh logic hoặc tính năng của đoạn mã là quá trình sửa đổi chương trình hiện có nhằm nâng cao hiệu suất, sửa lỗi hoặc thích ứng với yêu cầu mới.

Các loại thay đổi có thể bao gồm:

- Bảo trì và Sửa lỗi: Khắc phục lỗi thường gặp (như lỗi cú pháp, ngoại lệ) và cập nhật mã để đảm bảo chương trình chạy ổn định.
- Nâng cấp Chức năng: Thêm tính năng mới hoặc tùy chỉnh chương trình, ví dụ bổ sung hàm xử lý mới để tăng hiệu quả.

Sử dụng ChatGPT và Copilot để chỉnh sửa mã

Để sử dụng AI như ChatGPT hoặc GitHub Copilot để điều chỉnh logic hoặc tính năng của đoạn mã, cần cung cấp đoạn code hiện tại và mô tả thay đổi mong muốn, công cụ AI sẽ phân tích ngữ cảnh và đề xuất mã được chỉnh sửa hoặc thêm mới phù hợp nhằm nâng cao hiệu suất và tính năng.

- Phân tích ngữ cảnh chính xác: AI hiểu rõ cấu trúc và logic của đoạn mã hiện có để đưa ra các điều chỉnh phù hợp.
- Tiết kiệm thời gian: Tự động gợi ý sửa đổi hoặc bổ sung, giảm công sức viết lại mã thủ công.
- Nâng cao chất lượng mã: Đề xuất theo chuẩn coding và tối ưu hiệu suất, giúp mã nguồn dễ bảo trì hơn.

Sử dụng ChatGPT và Copilot để chỉnh gỡ lỗi

Gỡ lỗi (Debugging) là quá trình quan trọng giúp lập trình viên kiểm tra, phát hiện và sửa các lỗi trong mã nguồn thông qua thực thi chương trình nhiều lần để đảm bảo hoạt động chính xác.

- Xác định lỗi: Đọc thông báo lỗi hoặc quan sát kết quả đầu ra không đúng để phát hiện lỗi trong chương trình.
- Phân tích lỗi: Xem xét mã nguồn tại vị trí lỗi để hiểu nguyên nhân gây ra lỗi, có thể dùng trình gỡ lỗi (debugger) để theo dõi giá trị biến và luồng thực thi.
- Sửa lỗi và kiểm tra lại: Sửa mã nguồn tại vị trí lỗi, sau đó chạy lại chương trình nhiều lần với các dữ liệu kiểm thử khác nhau để đảm bảo lỗi đã được khắc phục hoàn toàn.

Sử dụng ChatGPT và Copilot để chỉnh gỡ lỗi

Để sử dụng AI như ChatGPT hoặc GitHub Copilot để tìm và sửa lỗi code, cần cung cấp phần code và mô tả lỗi, AI sẽ phân tích, phát hiện vấn đề và đề xuất chỉnh sửa tự động giúp tiết kiệm thời gian và cải thiện chất lượng mã.

- Phân tích lỗi nhanh chóng: AI hiểu ngữ cảnh và xác định lỗi cú pháp, logic hiệu quả.
- Đề xuất sửa lỗi chính xác: AI cung cấp giải pháp sửa trực tiếp hoặc viết lại đoạn mã tối ưu.
- Hỗ trợ liên tục: Công cụ AI tích hợp trực tiếp trong IDE giúp kiểm tra, sửa lỗi theo thời gian thực, tăng tốc quá trình phát triển.

Sử dụng ChatGPT và Copilot

Bài toán: Tính Giá trị Tương lai của số tiền đầu tư ban đầu với lãi suất kép trong một khoảng thời gian nhất định.

→ Prompt: “Viết đoạn mã Python tính Giá trị Tương lai (Future Value) dựa trên số tiền gốc, lãi suất hàng năm và số năm đầu tư theo công thức FV = P * (1 + r)^t. Đầu ra là đoạn mã Python hoàn chỉnh.”

Sử dụng ChatGPT và Copilot

Một số trường hợp có thể xảy ra:

- Lỗi nhập dữ liệu → ValueError
- Lỗi đơn vị lãi suất → kết quả sai
- Lỗi logic do số âm → kết quả vô lý
- Lỗi hiển thị → kết quả không đẹp
- Lỗi cú pháp ^ vs ** → sai hoàn toàn

→ Prompt: “Chỉnh sửa đoạn mã Python tính Giá trị Tương lai để cho phép người dùng nhập số tiền gốc, lãi suất và số năm từ bàn phím, đồng thời xử lý lỗi nhập liệu không hợp lệ bằng cách sử dụng xử lý ngoại lệ. Yêu cầu cung cấp mã Python hoàn chỉnh.”

Sử dụng ChatGPT và Copilot

Một số trường hợp có thể xảy ra:

- Lỗi nhập dữ liệu → ValueError
- Lỗi đơn vị lãi suất → kết quả sai
- Lỗi logic do số âm → kết quả vô lý
- Lỗi hiển thị → kết quả không đẹp
- Lỗi cú pháp ^ vs ** → sai hoàn toàn

→ Prompt: “Chỉnh sửa đoạn mã Python tính Giá trị Tương lai để cho phép người dùng nhập số tiền gốc, lãi suất và số năm từ bàn phím, đồng thời xử lý lỗi nhập liệu không hợp lệ bằng cách sử dụng xử lý ngoại lệ. Yêu cầu cung cấp mã Python hoàn chỉnh.”

Sử dụng ChatGPT và Copilot

Chạy 1 lần lỗi thì dừng luôn không xử lý tiếp được nữa

→ Prompt : “Đoạn mã Python tính Giá trị Tương lai bị lỗi khi nhập dữ liệu, hãy kiểm tra và sửa lỗi như ngoại lệ khi người dùng nhập giá trị không đúng kiểu, đảm bảo chương trình không bị dừng đột ngột. Đáp ứng yêu cầu bằng mã Python ổn định.”

Nội dung

Phần 3: Tự động hóa các Tác vụ Excel

1. Công thức nâng cao để tự động hóa
2. Giới thiệu về macros đơn giản
3. Nghiên cứu tình huống: Tự động hóa báo cáo tài chính hàng tháng

Công thức nâng cao

Công thức nâng cao trong Excel giúp xử lý dữ liệu phức tạp nhanh chóng và chính xác, tăng hiệu quả làm việc với các bảng tính lớn và nhiều điều kiện khác nhau.

- VLOOKUP: Tìm kiếm dữ liệu theo hàng trong bảng, cho phép lấy giá trị tương ứng dựa trên một giá trị khóa.
- INDEX/MATCH: Kết hợp dùng để tìm giá trị theo hàng và cột, linh hoạt hơn VLOOKUP, cho phép tìm kiếm hai chiều và xử lý bảng không cần sắp xếp.
- SUMIFS: Tính tổng các giá trị thỏa mãn nhiều điều kiện đồng thời, rất hữu ích trong phân tích số liệu phức tạp.

Macros

Macro trong Excel là những chương trình nhỏ được viết bằng VBA (Visual Basic for Applications), giúp tự động hóa các thao tác lặp đi lặp lại, từ đó tăng năng suất và giảm sai sót khi làm việc với dữ liệu phức tạp.

Ngoài ra, VBA cho phép người dùng tùy chỉnh chức năng Excel theo nhu cầu riêng và mở rộng khả năng phần mềm.

Macros

- Hiển thị tab Developer và nhấp Record Macro: Đảm bảo tab Developer đã được bật (thông qua File > Options > Customize Ribbon). Vào tab Developer, nhấp vào Record Macro để bắt đầu ghi lại thao tác.
- Đặt tên và thực hiện chuỗi hành động: Đặt tên cho Macro, có thể gán phím tắt (ví dụ Ctrl + t). Thực hiện liên tiếp các thao tác muốn tự động hóa, như chọn, sao chép, hoặc di chuyển đến ô khác với tham chiếu tương đối.
- Hoàn tất ghi Macro: Nhấp Stop Recording trên tab Developer để lưu Macro.

Thực hành macro

Yêu cầu: Quản lý dữ liệu bán hàng và phân tích hiệu quả kinh doanh hàng tháng của công ty bán lẻ. Cho thông tin sheet như sau: https://bom.so/nS7Jo5.

a. Làm sạch và chuẩn hóa dữ liệu (Data Cleaning)

Mục tiêu:

- Loại bỏ khoảng trắng dư thừa.
- Chuyển tên khách hàng, sản phẩm về dạng chữ hoa đầu dòng (Proper Case).
- Chuẩn hóa tên khu vực (Miền Bắc, Miền Trung, Miền Nam).

Thực hành macro

Yêu cầu: Quản lý dữ liệu bán hàng và phân tích hiệu quả kinh doanh hàng tháng của công ty bán lẻ. Cho thông tin sheet như sau: https://bom.so/nS7Jo5.

b. Tự động tạo báo cáo doanh số theo tháng và khu vực bằng Pivot Table.

Bố cục:

- Nguồn: Sheet Sales_Raw (đã làm sạch).
- Báo cáo tạo tại sheet Monthly_Report.

c. Giả lập các kịch bản "nếu giá bán tăng/giảm" hoặc "chi phí nhập thay đổi" thì lợi nhuận ra sao.

Ý tưởng:

- Dữ liệu tại sheet "Simulation".
- Người dùng nhập:
  + Giá bán hiện tại
  + Tỷ lệ tăng/giảm (%)
  + Chi phí
- VBA tự động tính lại lợi nhuận dự kiến.

Phần 3: Đọc và Phân tích các Tập dữ liệu Kinh tế nhỏ

1. Nhập tập dữ liệu vào Python hoặc Excel
2. Làm sạch và chuẩn bị dữ liệu
3. Tạo thống kê tóm tắt

Nhập tập dữ liệu vào Python hoặc Excel

Ba nguồn dữ liệu chính được nhập vào Python để phân tích bao gồm:

- Tệp Excel (.xlsx, .xls)
- Tệp CSV (Comma-Separated Values)
- API (Application Programming Interface)

→ Python là công cụ phổ biến cho phân tích dữ liệu lớn với Pandas, Openpyxl và xlrd, Requests, …

Làm việc với tập dữ liệu

1. Nhập dữ liệu CSV trong Python

Bước 1: Import thư viện csv hoặc pandas.

Bước 2: Mở file CSV.

Bước 3: Đọc dữ liệu bằng csv.reader hoặc pandas.read_csv.

Làm việc với tập dữ liệu

```
import csv
with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Hoặc với pandas
import pandas as pd
data = pd.read_csv('data.csv')
print(data.head())
```

2. Nhập dữ liệu Excel trong Python
Bước 1: Import pandas.
Bước 2: Đọc file Excel với tên file và tên sheet.

Ví dụ:

```
import pandas as pd
data = pd.read_excel('data.xlsx', sheet_name='Sheet1')
print(data.head())
```

3. Lấy dữ liệu từ API trong Python
Bước 1: Import các thư viện requests và json.
Bước 2: Gửi yêu cầu GET lấy dữ liệu từ API.
Bước 3: Xử lý dữ liệu JSON trả về.

Ví dụ:

```
import requests
response = requests.get('https://api.example.com/data')
data = response.json()
print(data)
```

4. Nhập dữ liệu vào Excel sử dụng Power Query

Bước 1: Mở Excel, chọn tab Data > Get Data.

Bước 2: Chọn From File > From CSV hoặc From Excel Workbook.

Bước 3: Dẫn đến tệp nguồn và tải dữ liệu.

Bước 4: (Tuỳ chọn) Chỉnh sửa, biến đổi dữ liệu trong Power Query Editor trước khi nạp vào workbook.

Làm sạch và chuẩn bị dữ liệu

Làm sạch và chuẩn bị dữ liệu là khía cạnh quan trọng trong quản lý dữ liệu, nhằm đảm bảo tính toàn vẹn và hiệu quả của dữ liệu.

Chuẩn bị dữ liệu bao gồm việc đồng nhất dữ liệu từ nhiều nguồn khác nhau qua các bước trích xuất, chuyển đổi và nạp vào kho dữ liệu để phục vụ phân tích (ETL).

Làm sạch và chuẩn bị dữ liệu

Dữ liệu cần được làm sạch khi gặp các vấn đề như:

- dư thừa dữ liệu (dữ liệu trùng lặp không cần thiết)
- không nhất quán (cùng dữ liệu có nhiều phiên bản khác nhau)
- dữ liệu không hợp lệ hoặc không phù hợp với quy tắc logic,
- dữ liệu không đầy đủ (trường bắt buộc bị bỏ trống)
- định dạng không nhất quán giữa các bản ghi.

Làm sạch và chuẩn bị dữ liệu

Các bước cơ bản trong làm sạch và chuẩn bị dữ liệu

1. Xác thực và Ngăn ngừa Lỗi (Validation and Error Prevention)
   - Kiểm tra phạm vi: Đảm bảo dữ liệu nằm trong giới hạn giá trị cho phép.
   - Kiểm tra tính đầy đủ: Các trường bắt buộc phải có dữ liệu.
   - Kiểm tra tính nhất quán: So sánh dữ liệu giữa các trường để đảm bảo hợp lý (ví dụ: ngày sinh trước ngày nhập học).
   - Định dạng trường: Tự động áp dụng mẫu định dạng chuẩn (ví dụ: định dạng số điện thoại).

Làm sạch và chuẩn bị dữ liệu

2. Chuẩn hóa và Tái cấu trúc (Normalization and Restructuring):
   - Chuẩn hóa: Loại bỏ dư thừa và không nhất quán bằng cách phân tách bảng lớn thành các bảng nhỏ có liên kết.
   - Phân tách trường: Tách dữ liệu thành các phần nhỏ hơn (ví dụ: tách họ và tên riêng).
   - Cập nhật toàn cục: Dùng lệnh SQL UPDATE để thay đổi đồng loạt các bản ghi sai lệch.
   - Tìm kiếm và thay thế: Sửa lỗi lặp lại bằng công cụ tìm kiếm và thay thế trên dữ liệu.

Thống kê Tóm tắt

Các chỉ số thống kê tóm tắt chính gồm Mean, Median và Standard Deviation giúp mô tả đặc trưng trung bình và sự phân tán của dữ liệu trong bộ dữ liệu.

- Mean (Trung bình cộng): Là tổng các giá trị chia cho số lượng giá trị.
- Median (Trung vị): Là giá trị đứng chính giữa khi dữ liệu được sắp xếp theo thứ tự, phản ánh giá trị trung tâm.
- Standard Deviation (Độ lệch chuẩn): Là căn bậc hai của phương sai, đo mức độ phân tán hoặc biến thiên của dữ liệu quanh giá trị trung bình.

THANK YOU FOR YOUR ATTENTION!